AMRITA VISHWA VIDYAPEETHAM

Name: Ganath Avinash G.R

CSE – B

CH.SC.U4CSE24118

Week –3

1. BFS

BFS.c > ...

```c
#include <stdio.h>

int queue[20], front = -1, rear = -1;
int visited[20];

void bfs(int graph[20][20], int n, int start) {
    queue[++rear] = start;
    visited[start] = 1;

    while (front != rear) {
        int v = queue[++front];
        printf("%d ", v);

        for (int i = 0; i < n; i++) {
            if (graph[v][i] == 1 && visited[i] == 0) {
                queue[++rear] = i;
                visited[i] = 1;
            }
        }
    }
}

int main() {
    int n = 4;
    int graph[20][20] = {
        {0,1,1,0},
        {1,0,0,1},
        {1,0,0,1},
        {0,1,1,0}
    };

    bfs(graph, n, 0);
    return 0;
}
```

OUTPUT:

```
PS C:\Users\Ganath Avinash\OneDrive\ドキュメント\Back-end\DAA> cd
0 1 2 3
PS C:\Users\Ganath Avinash\OneDrive\ドキュメント\Back-end\DAA>
```

Space Complexity:O(V)
Time Complexity:O(V+E)

Justification:

The queue can store upto V vertices in the worst case. Hence the Space Complexity is O(V).
BFS visits every vertex once and checks every edge once while exploring adjacency lists .Hence the Time Complexity is O(V+E)

2. DFS

```c
DFS.c > ...
#include <stdio.h>

int visited[20];

void dfs(int graph[20][20], int n, int v) {
    printf("%d ", v);
    visited[v] = 1;

    for (int i = 0; i < n; i++) {
        if (graph[v][i] == 1 && visited[i] == 0) {
            dfs(graph, n, i);
        }
    }
}

int main() {
    int n = 4;
    int graph[20][20] = {
        {0,1,1,0},
        {1,0,0,1},
        {1,0,0,1},
        {0,1,1,0}
    };

    dfs(graph, n, 0);
    return 0;
}
```

OUTPUT:

```
amma@amma15:~/Documents/CH.SC.U4CSE24217$ gcc -o is inserts.c
amma@amma15:~/Documents/CH.SC.U4CSE24217$ ./is
Insertion Sorted O(n^2):
 2  2  3  3  4  4  4  4  5  5  7  11  12  245  567
amma@amma15:~/Documents/CH.SC.U4CSE24217$
```

Space Complexity:O(V)
Time Complexity:O(V+E)

Justification:

The recursion stack can grow upto V levels and the visited [ ] array stores V entries. Hence the Space Complexity is O(V).
DFS recursively explores every vertex and inspects all edges exactly once through the adjacency list. Hence the Time Complexity is O(V+E).